

Министерство науки и высшего образования Российской
Федерации федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
Институт информатики и кибернетики

Отчет по лабораторной работе № 7

Дисциплина: «ООП»

Тема «Паттерны проектирования и рефлексия»

Выполнил: Сучков Борис Антонович

Группа: 6201-120303D

Задание на лабораторную работу

Внести изменения в существующий набор типов табулированных функций, позволяющие обрабатывать точки функций по порядку (паттерн «Итератор»), а также выбирать тип объекта табулированной функции при его неявном создании (паттерн «Фабричный метод» и средства рефлексии).

Задание 1

Сделайте так, чтобы все объекты типа `TabulatedFunction` можно было использовать в качестве объекта-агрегата в «улучшенном цикле `for`» (вариант `for-each`), извлекаемые объекты при этом должны иметь тип `FunctionPoint`.

В интерфейсе `TabulatedFunction` добавьте необходимый родительский тип, используйте при этом параметризованный тип (generic type).

В классах, реализующих интерфейс `TabulatedFunction`, добавьте требуемый метод, возвращающий объект итератора.

Классы итераторов сделайте анонимными. В соответствии с паттерном «Итератор» итераторы должны работать эффективно и использовать знание о внутренней структуре объектов, а не вызывать публичные методы объекта табулированной функции.

Операцию удаления текущего элемента в итераторах реализовывать не нужно. Метод удаления должен всегда выбрасывать исключение `UnsupportedOperationException`.

Метод получения следующего элемента должен выбрасывать исключение `NoSuchElementException`, если следующего элемента нет. Возвращаемый методом объект типа `FunctionPoint` не должен позволять нарушить инкапсуляцию объекта табулированной функции.

В методе `main()` программы проверьте работу итераторов классов табулированных функций, например, следующим образом.

```
TabulatedFunction f = /× получение или создание объекта ×/; for
(FunctionPoint p : f) {
    System.out.println(p);
}
```

Задание 2

В классе `TabulatedFunctions` ряд методов в ходе своей работы порождает объекты табулированных функций. При этом реальный тип объектов оказывался фиксированным, и изменить его динамически в ходе работы программы было невозможно. Одним из способов решения этой проблемы является применение объектов фабрик с возможностью их замены. Поскольку здесь требуется порождение только одного объекта, воспользуемся паттерном «Фабричный метод». В пакете `functions` опишите базовый интерфейс фабрик табулированных функций `TabulatedFunctionFactory`. Интерфейс должен объявлять три перегруженных метода

`TabulatedFunction createTabulatedFunction()`, параметры которых соответствуют параметрам конструкторов классов табулированных функций.

Для удобства (вообще говоря, такой элемент дизайна необязателен) опишите в классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` классы фабрик

`ArrayTabulatedFunctionFactory` и `LinkedListTabulatedFunctionFactory`, реализующие интерфейс фабрики и порождающие объекты соответствующих классов табулированных функций. Сделайте классы фабрик вложенными и публичными. В классе `TabulatedFunctions` объявите приватное статическое поле типа `TabulatedFunctionFactory` и проинициализируйте его объектом одного из описанных классов фабрик. Также объявите метод `setTabulatedFunctionFactory()`, позволяющий заменить объект фабрики.

Ещё в классе `TabulatedFunctions` опишите три перегруженных метода

`TabulatedFunction createTabulatedFunction()`, возвращающих объекты табулированных функций, созданные с помощью текущей фабрики. Параметры методов должны соответствовать параметрам методов фабрики.

В остальных методах класса, где требуется создание объектов табулированных функций, замените явное создание объектов с помощью конструкторов на вызов соответствующего метода `createTabulatedFunction()`.

В методе `main()` проверьте работу фабрик, например, следующим образом.

```
Function f = new Cos(); TabulatedFunction
tf;
tf = TabulatedFunctions.tabulate(f, 0, Math.PI, 11);
System.out.println(tf.getClass());
TabulatedFunctions.setTabulatedFunctionFactory(new
    LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f, 0, Math.PI, 11);
System.out.println(tf.getClass());
TabulatedFunctions.setTabulatedFunctionFactory(new
    ArrayTabulatedFunction.ArrayTabulatedFunctionFactory()); tf
= TabulatedFunctions.tabulate(f, 0, Math.PI, 11);
System.out.println(tf.getClass());
```

Задание 3

Ещё одним инструментом, позволяющим указывать типы порождаемых объектов, является рефлексия.

В классе `TabulatedFunctions` добавьте ещё три перегруженных версии метода `createTabulatedFunction()`. Их параметры должны повторять параметры трёх аналогичных методов, основанных на использовании фабрики, но также эти методы должны получать ссылку типа `Class` на описание класса, объект которого требуется создать. Сделайте так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс `TabulatedFunction`.

Новые методы создания объектов должны найти в предложенном классе конструктор с соответствующими типами параметров (например, двумя параметрами типа `double` и одним параметром типа `int` для метода, создающего объект табулированной функции по левой и правой границе области определения и количеству точек). С помощью

найденного конструктора (в него должны быть переданы фактические параметры) должен быть создан объект табулированной функции. Ссылка на этот объект и должна быть возвращена из метода создания.

Если в ходе выполнения рефлексивных операций возникло исключение (не найден конструктор и т.д.), оно должно быть отловлено (используйте блок try с отловом нескольких типов исключений). Вместо него должно быть выброшено исключение `IllegalArgumentException`, причём в его конструктор должно быть передано отловленное исключение из рефлексии. Это позволит в случае возникновения ошибок определить реальную причину ошибки.

В классе `TabulatedFunctions` перегрузите методы, создающие объекты табулированных функций, добавив версии, принимающие также ссылку типа `Class` на описание класса, объект которого требуется создать. Сделайте так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс `TabulatedFunction`.

Проверьте в методе `main()` работу методов рефлексивного создания объектов, а также методов класса `TabulatedFunctions`, использующих создание объектов. Это можно сделать, например, следующим образом.

```
TabulatedFunction f;  
f =  
TabulatedFunctions.createTabulatedFunction(  
    ArrayTabulatedFunction.class, 0, 10, 3);  
System.out.println(f.getClass());  
System.out.println(f);  
  
f = TabulatedFunctions.createTabulatedFunction(  
    ArrayTabulatedFunction.class, 0, 10, new double[] {0, 10});  
System.out.println(f.getClass());  
System.out.println(f);  
  
f = TabulatedFunctions.createTabulatedFunction(  
    LinkedListTabulatedFunction.class,  
    new FunctionPoint[] {      new  
        FunctionPoint(0, 0),    new  
        FunctionPoint(10, 10)  
    }  
);  
System.out.println(f.getClass());  
System.out.println(f);  
  
f = TabulatedFunctions.tabulate(  
    LinkedListTabulatedFunction.class, new Sin(), 0, Math.PI, 11);  
System.out.println(f.getClass());  
System.out.println(f);
```

Задание 1

Я сделайте так, что все объекты типа `TabulatedFunction` можно использовать в качестве объекта-агрегата в «улучшенном цикле for» (вариант `for-each`), извлекаемые объекты при этом имеют тип `FunctionPoint`.

В интерфейсе `TabulatedFunction` я добавил необходимый родительский тип, использовал при этом параметризованный тип (generic type).

В классах, реализующих интерфейс `TabulatedFunction`, я добавил требующийся метод, возвращающий объект итератора.

Классы итераторов сделал анонимными. В соответствии с паттерном «Итератор» итераторы работают эффективно и используют знание о внутренней структуре объектов, а не вызывают публичные методы объекта табулированной функции.

Метод удаления всегда выбрасывает исключение `UnsupportedOperationException`.

Метод получения следующего элемента выбрасывает исключение `NoSuchElementException`, если следующего элемента нет. Возвращаемый методом объект типа `FunctionPoint` не позволяет нарушить инкапсуляцию объекта табулированной функции.

В методе `main()` программы я проверил работу итераторов классов табулированных функций.

```
TabulatedFunction.java M X
src > functions > TabulatedFunction.java > ...
1 package functions;
2
3 // Наследуемся от Iterable<FunctionPoint>, чтобы можно было использовать в цикле for-each
4 public interface TabulatedFunction extends Function, Cloneable, java.io.Serializable, Iterable<FunctionPoint> {
5     int getPointsCount();
6     FunctionPoint getPoint(int index);
7     void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;
8     double getPointX(int index);
9     void setPointX(int index, double x) throws InappropriateFunctionPointException;
10    double getPointY(int index);
11    void setPointY(int index, double y);
12    void deletePoint(int index);
13    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
14
15    Object clone() throws CloneNotSupportedException;
16 }
17
```

```

1 package functions;
2
3 import java.util.Arrays;
4 import java.util.Iterator;
5 import java.util.NoSuchElementException;
6
7 public class ArrayTabulatedFunction implements TabulatedFunction {
8     private FunctionPoint[] points; // Массив точек
9     private static final long serialVersionUID = 1L;
10
11     // --- Конструкторы ---
12
13     // Создание по границам и количеству точек (значения Y = 0)
14     public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {
15         if (leftX >= rightX || pointsCount < 2) {
16             throw new IllegalArgumentException(s: "Invalid arguments: leftX >= rightX or pointsCount < 2");
17         }
18         this.points = new FunctionPoint(pointsCount);
19         double step = (rightX - leftX) / (pointsCount - 1);
20         for (int i = 0; i < pointsCount; ++i) {
21             points[i] = new FunctionPoint(leftX + i * step, y: 0);
22         }
23     }
24
25     // Создание по границам и массиву значений Y
26     public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {
27         int count = values.length;
28         if (leftX >= rightX || count < 2) {
29             throw new IllegalArgumentException(s: "Invalid arguments: leftX >= rightX or pointsCount < 2");
30         }
31         this.points = new FunctionPoint(count);
32         double step = (rightX - leftX) / (count - 1);
33         for (int i = 0; i < count; ++i) {
34             points[i] = new FunctionPoint(leftX + i * step, values[i]);
35         }
36     }
37
38     // Создание из готового массива точек
39     public ArrayTabulatedFunction(FunctionPoint[] points) {
40         if (points.length < 2) {
41             throw new IllegalArgumentException(s: "Function must have at least 2 points");
42         }
43         // Проверка на сортировку
44         for (int i = 0; i < points.length - 1; ++i) {
45             if (points[i].getX() >= points[i + 1].getX()) {
46                 throw new IllegalArgumentException(s: "Points are not sorted by X");
47             }
48         }
49         // Копирование точек (глубокое копирование)
50         this.points = new FunctionPoint(points.length);
51         for (int i = 0; i < points.length; ++i) {
52             this.points[i] = new FunctionPoint(points[i]);
53         }
54     }
55
56     // --- Основные методы ---
57
58     public double getLeftDomainBorder() {

```

```

59         return points[0].getX();
60     }
61
62     public double getRightDomainBorder() {
63         return points[points.length - 1].getX();
64     }
65
66     // Вычисление значения функции (Интерполяция)
67     public double getFunctionValue(double x) {
68         if (x < getLeftDomainBorder() || x > getRightDomainBorder()) {
69             return Double.NaN;
70         }
71         for (int i = 0; i < points.length - 1; ++i) {
72             double x1 = points[i].getX();
73             double x2 = points[i + 1].getX();
74             double y1 = points[i].getY();
75             double y2 = points[i + 1].getY();
76
77             if (x >= x1 && x <= x2) {
78                 double epsilon = 1e-9;
79                 if (Math.abs(x - x1) < epsilon) return y1;
80                 if (Math.abs(x - x2) < epsilon) return y2;
81                 // Линейная интерполяция
82                 return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
83             }
84         }
85         return Double.NaN;
86     }
87
88     public int getPointsCount() { return points.length; }
89
90     // Проверка корректности индекса
91     private void checkIndex(int index) {
92         if (index < 0 || index >= points.length) {
93             throw new FunctionPointIndexOutOfBoundsException("Index " + index + " is out of bounds");
94         }
95     }
96
97     public FunctionPoint getPoint(int index) {
98         checkIndex(index);
99         return new FunctionPoint(points[index]);
100     }
101
102     public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
103         checkIndex(index);
104         double newX = point.getX();
105         // Проверка, что новый X не нарушает порядок сортировки
106         double leftBound = (index > 0) ? points[index - 1].getX() : Double.NEGATIVE_INFINITY;
107         double rightBound = (index < points.length - 1) ? points[index + 1].getX() : Double.POSITIVE_INFINITY;
108
109         if (newX <= leftBound || newX >= rightBound) {
110             throw new InappropriateFunctionPointException(message: "New point's X coordinate is out of the allowed interval");
111         }
112         points[index] = new FunctionPoint(point);
113     }

```



```

ArrayTabulatedFunction.java in X
src > functions > ArrayTabulatedFunction.java >
7 public class ArrayTabulatedFunction implements TabulatedFunction {
113 }
114
115 public double getPointX(int index) {
116     checkIndex(index);
117     return points[index].getX();
118 }
119
120 public void setPointX(int index, double x) throws InappropriateFunctionPointException {
121     checkIndex(index);
122     // Проверка границ для X
123     double leftBound = (index > 0) ? points[index - 1].getX() : Double.NEGATIVE_INFINITY;
124     double rightBound = (index < points.length - 1) ? points[index + 1].getX() : Double.POSITIVE_INFINITY;
125
126     if (x <= leftBound || x >= rightBound) {
127         throw new InappropriateFunctionPointException(message: "New X coordinate is out of the allowed interval");
128     }
129     points[index].setX(x);
130 }
131
132 public double getPointY(int index) {
133     checkIndex(index);
134     return points[index].getY();
135 }
136
137 public void setPointY(int index, double y) {
138     checkIndex(index);
139     points[index].setY(y);
140 }
141
142 // Удаление точки (сдвиг массива)
143 public void deletePoint(int index) {
144     checkIndex(index);
145     if (points.length < 3) {
146         throw new IllegalStateException(s: "Cannot delete point: function must have at least 2 points remaining.");
147     }
148     FunctionPoint[] newPoints = new FunctionPoint[points.length - 1];
149     System.arraycopy(points, srcPos: 0, newPoints, destPos: 0, index);
150     System.arraycopy(points, index + 1, newPoints, index, points.length - index - 1);
151     points = newPoints;
152 }
153
154 // Добавление точки (создание нового массива)
155 public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
156     int insertIndex = 0;
157     // Поиск места для вставки
158     while (insertIndex < points.length && points[insertIndex].getX() < point.getX()) {
159         insertIndex++;
160     }
161     if (insertIndex < points.length && points[insertIndex].getX() == point.getX()) {
162         throw new InappropriateFunctionPointException("Point with X=" + point.getX() + " already exists.");
163     }
164     FunctionPoint[] newPoints = new FunctionPoint[points.length + 1];
165     System.arraycopy(points, srcPos: 0, newPoints, destPos: 0, insertIndex);
166     newPoints[insertIndex] = new FunctionPoint(point);
167     System.arraycopy(points, insertIndex, newPoints, insertIndex + 1, points.length - insertIndex);
168     points = newPoints;
169 }

```

```

ArrayTabulatedFunction.java in X
src > functions > ArrayTabulatedFunction.java >
7 public class ArrayTabulatedFunction implements TabulatedFunction {
155 public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
165     System.arraycopy(points, srcPos: 0, newPoints, destPos: 0, insertIndex);
166     newPoints[insertIndex] = new FunctionPoint(point);
167     System.arraycopy(points, insertIndex, newPoints, insertIndex + 1, points.length - insertIndex);
168     points = newPoints;
169 }
170
171 public String toString() {
172     StringBuilder sb = new StringBuilder();
173     sb.append(str: "(");
174     for (int i = 0; i < points.length; i++) {
175         sb.append(points[i].toString());
176         if (i < points.length - 1) {
177             sb.append(str: ", ");
178         }
179     }
180     sb.append(str: ")");
181     return sb.toString();
182 }
183
184 public boolean equals(Object o) {
185     if (this == o) return true;
186     if (!(o instanceof TabulatedFunction)) return false;
187
188     TabulatedFunction that = (TabulatedFunction) o;
189     if (this.getPointsCount() != that.getPointsCount()) return false;
190
191     if (o instanceof ArrayTabulatedFunction) {
192         ArrayTabulatedFunction thatArray = (ArrayTabulatedFunction) o;
193         for (int i = 0; i < this.points.length; i++) {
194             if (!this.points[i].equals(thatArray.points[i])) return false;
195         }
196     } else {
197         for (int i = 0; i < this.getPointsCount(); i++) {
198             if (!this.getPoint(i).equals(that.getPoint(i))) return false;
199         }
200     }
201     return true;
202 }
203
204 public int hashCode() {
205     int result = Arrays.hashCode(points);
206     result = 31 * result + Integer.hashCode(points.length);
207     return result;
208 }
209
210 public Object clone() throws CloneNotSupportedException {
211     FunctionPoint[] clonedPoints = new FunctionPoint[this.points.length];
212     for (int i = 0; i < this.points.length; i++) {
213         clonedPoints[i] = (FunctionPoint) this.points[i].clone();
214     }
215     return new ArrayTabulatedFunction(clonedPoints);
216 }
217
218 // --- ЗАДАНИЕ 1: Итератор ---
219 public Iterator<FunctionPoint> iterator() {
220     return new Iterator<FunctionPoint>() {

```

```

205     int result = Arrays.hashCode(points);
206     result = 31 * result + Integer.hashCode(points.length);
207     return result;
208 }
209
210 public Object clone() throws CloneNotSupportedException {
211     FunctionPoint[] clonedPoints = new FunctionPoint[this.points.length];
212     for (int i = 0; i < this.points.length; i++) {
213         clonedPoints[i] = (FunctionPoint) this.points[i].clone();
214     }
215     return new ArrayTabulatedFunction(clonedPoints);
216 }
217
218 // --- Итератор ---
219 public Iterator<FunctionPoint> iterator() {
220     return new Iterator<FunctionPoint>() {
221         private int index = 0;
222
223         public boolean hasNext() {
224             return index < points.length;
225         }
226
227         public FunctionPoint next() {
228             if (!hasNext()) {
229                 throw new NoSuchElementException();
230             }
231             // Возвращаем копию точки
232             return new FunctionPoint(points[index++]);
233         }
234
235         public void remove() {
236             throw new UnsupportedOperationException();
237         }
238     };
239 }
240
241 }

```

```

src > functions > LinkedTabulatedFunction.java > ...
1 package functions;
2
3 import java.io.Externalizable;
4 import java.io.IOException;
5 import java.io.ObjectInput;
6 import java.io.ObjectOutput;
7 import java.util.Iterator;
8 import java.util.NoSuchElementException;
9
10 public class LinkedListTabulatedFunction implements TabulatedFunction, Externalizable {
11
12     // Внутренний класс узла списка
13     private class FunctionNode {
14         FunctionPoint point;
15         FunctionNode prev;
16         FunctionNode next;
17
18         FunctionNode() {}
19     }
20
21     private FunctionNode head; // Голова списка (фиктивный элемент)
22     private int count; // Количество элементов
23     private static final long serialVersionUID = 1L;
24
25     // --- Конструкторы ---
26
27     // Пустой конструктор (создает только голову)
28     public LinkedListTabulatedFunction() {
29         this.count = 0;
30         this.head = new FunctionNode();
31         this.head.prev = head;
32         this.head.next = head;
33     }
34
35     // Конструктор по границам и количеству точек
36     public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {
37         if (leftX >= rightX || pointsCount < 2) {
38             throw new IllegalArgumentException("Invalid arguments for function creation");
39         }
40         this.count = 0;
41         this.head = new FunctionNode();
42         this.head.prev = head;
43         this.head.next = head;
44
45         double step = (rightX - leftX) / (pointsCount - 1);
46         for (int i = 0; i < pointsCount; i++) {
47             try {
48                 this.addPoint(new FunctionPoint(leftX + i * step, y: 0));
49             } catch (InappropriateFunctionPointException e) { /* Невозможно */ }
50         }
51     }
52
53     // Конструктор по массиву значений
54     public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
55         if (leftX >= rightX || values.length < 2) {
56             throw new IllegalArgumentException("Invalid arguments for function creation");
57         }
58         this.count = 0;

```



```

10 public class LinkedListTabulatedFunction implements TabulatedFunction, Externalizable {
54     public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
55         this.leftX = leftX;
56         this.rightX = rightX;
57         this.values = values;
58         this.head = new FunctionNode();
59         this.head.prev = head;
60         this.head.next = head;
61
62         double step = (rightX - leftX) / (values.length - 1);
63         for (int i = 0; i < values.length; i++) {
64             try {
65                 this.addPoint(new FunctionPoint(leftX + i * step, values[i]));
66             } catch (InappropriateFunctionPointException e) { /* Невозможно */ }
67         }
68     }
69
70     // Конструктор по массиву точек
71     public LinkedListTabulatedFunction(FunctionPoint[] points) {
72         if (points.length < 2) {
73             throw new IllegalArgumentException(s: "Function must have at least 2 points");
74         }
75         this.count = 0;
76         this.head = new FunctionNode();
77         this.head.prev = head;
78         this.head.next = head;
79
80         for (int i = 0; i < points.length; ++i) {
81             if (i > 0 && points[i-1].getX() >= points[i].getX()) {
82                 throw new IllegalArgumentException(s: "Points are not sorted by X");
83             }
84             try {
85                 this.addPoint(new FunctionPoint(points[i]));
86             } catch (InappropriateFunctionPointException e) { /* Невозможно */ }
87         }
88     }
89
90     // --- Внутренние методы ---
91
92     // Получение узла по индексу (оптимизация поиска с начала или конца)
93     private FunctionNode getNodeByIndex(int index) {
94         if (index < 0 || index >= count) {
95             throw new FunctionPointIndexOutOfBoundsException("Index " + index + " is out of bounds");
96         }
97         FunctionNode current;
98         if (index < count / 2) {
99             current = head.next;
100             for (int i = 0; i < index; i++) {
101                 current = current.next;
102             }
103         } else {
104             current = head.prev;
105             for (int i = count - 1; i > index; i--) {
106                 current = current.prev;
107             }
108         }
109         return current;
110     }
111
112     // Добавление узла в список
113

```

```

113 // Добавление узла в список
114 private FunctionNode addNode(FunctionNode node) {
115     FunctionNode newNode = new FunctionNode();
116     newNode.prev = node.prev;
117     newNode.next = node;
118     node.prev.next = newNode;
119     node.prev = newNode;
120     count++;
121     return newNode;
122 }
123
124 // Удаление узла из списка
125 private FunctionNode deleteNode(FunctionNode node) {
126     node.prev.next = node.next;
127     node.next.prev = node.prev;
128     count--;
129     return node;
130 }
131
132 // --- Основные методы ---
133
134 public double getLeftDomainBorder() { return head.next.point.getX(); }
135 public double getRightDomainBorder() { return head.prev.point.getX(); }
136
137 public double getFunctionValue(double x) {
138     if (x < getLeftDomainBorder() || x > getRightDomainBorder() || count == 0) {
139         return Double.NaN;
140     }
141
142     FunctionNode current = head.next;
143     while (current != head) {
144         double epsilon = 1e-9;
145         if (Math.abs(current.point.getX() - x) < epsilon) {
146             return current.point.getY();
147         }
148         if (current.next != head) {
149             double x1 = current.point.getX();
150             double x2 = current.next.point.getX();
151             if (x > x1 && x < x2) {
152                 if (Math.abs(x2 - x) < epsilon) return current.next.point.getY();
153                 double y1 = current.point.getY();
154                 double y2 = current.next.point.getY();
155                 // Интерполяция
156                 return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
157             }
158         }
159         current = current.next;
160     }
161     return Double.NaN;
162 }
163
164 public int getPointsCount() { return count; }
165
166 private void checkIndex(int index) {
167     if (index < 0 || index >= count) {
168         throw new FunctionPointIndexOutOfBoundsException("Index " + index + " is out of bounds (0, " + (count - 1) + ")");
169     }
170 }

```

```

10 public class LinkedListTabulatedFunction implements TabulatedFunction, Externalizable {
166 private void checkIndex(int index) {
169 }
170 }
171
172 public FunctionPoint getPoint(int index) {
173     checkIndex(index);
174     return new FunctionPoint(getNodeByIndex(index).point);
175 }
176
177 public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
178     checkIndex(index);
179     FunctionNode node = getNodeByIndex(index);
180     double newX = point.getX();
181     double leftBound = (node.prev == head) ? Double.NEGATIVE_INFINITY : node.prev.point.getX();
182     double rightBound = (node.next == head) ? Double.POSITIVE_INFINITY : node.next.point.getX();
183
184     if (newX <= leftBound || newX >= rightBound) {
185         throw new InappropriateFunctionPointException(message: "X coordinate is out of the allowed interval");
186     }
187     node.point = new FunctionPoint(point);
188 }
189
190 public double getPointX(int index) {
191     checkIndex(index);
192     return getNodeByIndex(index).point.getX();
193 }
194
195 public void setPointX(int index, double x) throws InappropriateFunctionPointException {
196     checkIndex(index);
197     FunctionNode node = getNodeByIndex(index);
198     double leftBound = (node.prev == head) ? Double.NEGATIVE_INFINITY : node.prev.point.getX();
199     double rightBound = (node.next == head) ? Double.POSITIVE_INFINITY : node.next.point.getX();
200
201     if (x <= leftBound || x >= rightBound) {
202         throw new InappropriateFunctionPointException(message: "X coordinate is out of the allowed interval");
203     }
204     node.point.setX(x);
205 }
206
207 public double getPointY(int index) {
208     checkIndex(index);
209     return getNodeByIndex(index).point.getY();
210 }
211
212 public void setPointY(int index, double y) {
213     checkIndex(index);
214     getNodeByIndex(index).point.setY(y);
215 }
216
217 public void deletePoint(int index) {
218     checkIndex(index);
219     if (count < 3) {
220         throw new IllegalStateException(s: "Cannot delete point: function must have at least 2 points remaining.");
221     }
222     deleteNode(getNodeByIndex(index));
223 }
224
225
226
227
228
229
230
231
232
233
234
235
236 // Сериализация (Externalizable)
237 public void writeExternal(ObjectOutput out) throws IOException {
238     out.writeInt(count);
239     FunctionNode current = head.next;
240     while (current != head) {
241         out.writeObject(current.point);
242         current = current.next;
243     }
244 }
245
246 // Десериализация
247 public void readExternal(ObjectInput in) throws IOException, ClassNotFoundException {
248     int readCount = in.readInt();
249     this.head.prev = head;
250     this.head.next = head;
251     this.count = 0;
252
253     for (int i = 0; i < readCount; i++) {
254         FunctionPoint point = (FunctionPoint) in.readObject();
255         try {
256             this.addPoint(point);
257         } catch (InappropriateFunctionPointException e) {
258             throw new IOException("Failed to deserialize: " + e.getMessage());
259         }
260     }
261 }
262
263 public String toString() {
264     StringBuilder sb = new StringBuilder();
265     sb.append(str: "{");
266     FunctionNode current = head.next;
267     while (current != head) {
268         sb.append(current.point.toString());
269         if (current.next != head) {
270             sb.append(str: ", ");
271         }
272         current = current.next;
273     }
274     sb.append(str: "}");
275     return sb.toString();
276 }
277
278 public boolean equals(Object o) {
279     if (this == o) return true;

```

```

src > functions > | LinkedListTabulatedFunction.java > ...
10 public class LinkedListTabulatedFunction implements TabulatedFunction, Externalizable {
263     public String toString() {
276     }
277
278     public boolean equals(Object o) {
279         if (this == o) return true;
280         if (!(o instanceof TabulatedFunction)) return false;
281
282         TabulatedFunction that = (TabulatedFunction) o;
283         if (this.getPointsCount() != that.getPointsCount()) return false;
284
285         if (o instanceof LinkedListTabulatedFunction) {
286             LinkedListTabulatedFunction thatList = (LinkedListTabulatedFunction) o;
287             FunctionNode thisCurrent = this.head.next;
288             FunctionNode thatCurrent = thatList.head.next;
289             while (thisCurrent != this.head) {
290                 if (!thisCurrent.point.equals(thatCurrent.point)) return false;
291                 thisCurrent = thisCurrent.next;
292                 thatCurrent = thatCurrent.next;
293             }
294         } else {
295             for (int i = 0; i < this.getPointsCount(); i++) {
296                 if (!this.getPoint(i).equals(that.getPoint(i))) return false;
297             }
298         }
299         return true;
300     }
301
302     public int hashCode() {
303         int result = 1;
304         FunctionNode current = head.next;
305         while (current != head) {
306             result = 31 * result + current.point.hashCode();
307             current = current.next;
308         }
309         result = 31 * result + Integer.hashCode(count);
310         return result;
311     }
312
313     public Object clone() throws CloneNotSupportedException {
314         FunctionPoint[] clonedPoints = new FunctionPoint[this.count];
315         FunctionNode current = this.head.next;
316         for (int i = 0; i < this.count; i++) {
317             clonedPoints[i] = (FunctionPoint) current.point.clone();
318             current = current.next;
319         }
320         return new LinkedListTabulatedFunction(clonedPoints);
321     }
322
323     // --- ЗАДАНИЕ 1: Итератор ---
324     public Iterator<FunctionPoint> iterator() {
325         return new Iterator<FunctionPoint>() {
326             private FunctionNode node = head.next; // Начинаем с первого реального элемента
327
328             public boolean hasNext() {
329                 // Если node == head, значит мы прошли весь круг
330                 return node != head;
331             }
332
333             public FunctionPoint next() {
334                 if (!hasNext()) {
335                     throw new NoSuchElementException();
336                 }
337                 FunctionPoint point = new FunctionPoint(node.point);
338                 node = node.next; // Переход к следующему
339                 return point;
340             }
341
342             public void remove() {
343                 throw new UnsupportedOperationException();
344             }
345         };
346     }
347
348 }

```

Задание 2

В классе `TabulatedFunctions` ряд методов в ходе своей работы порождает объекты табулированных функций. При этом реальный тип объектов оказывался фиксированным, и изменить его динамически в ходе работы программы было невозможно. Одним из способов решения этой проблемы является применение объектов фабрик с возможностью их замены. Поскольку здесь требуется порождение только одного объекта, воспользуемся паттерном «Фабричный метод». В пакете `functions` я описал базовый интерфейс фабрик табулированных функций `TabulatedFunctionFactory`. Интерфейс объявляет три перегруженных метода `TabulatedFunction createTabulatedFunction()`, параметры которых соответствуют параметрам конструкторов классов табулированных функций. Для удобства я описал в классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` классы фабрик

ArrayTabulatedFunctionFactory и LinkedListTabulatedFunctionFactory, реализующие интерфейс фабрики и порождающие объекты соответствующих классов табулированных функций. Я сделал классы фабрик вложенными и публичными. В классе TabulatedFunctions я объявил приватное статическое поле типа TabulatedFunctionFactory и проинициализировал его объектом одного из описанных классов фабрик. Также я объявил метод setTabulatedFunctionFactory(), позволяющий заменить объект фабрики.

Ещё в классе TabulatedFunctions я описал три перегруженных метода TabulatedFunction createTabulatedFunction(), возвращающих объекты табулированных функций, созданные с помощью текущей фабрики. Параметры методов соответствуют параметрам методов фабрики.

В остальных методах класса, где требуется создание объектов табулированных функций, я заменил явное создание объектов с помощью конструкторов на вызов соответствующего метода createTabulatedFunction().

В методе main() я проверил работу фабрик.

```
TabulatedFunctionFactory.java U X
src > functions > TabulatedFunctionFactory.java > ...
1 package functions;
2
3 // Интерфейс фабрики для создания табулированных функций
4 public interface TabulatedFunctionFactory {
5     TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);
6     TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);
7     TabulatedFunction createTabulatedFunction(FunctionPoint[] points);
8 }
9
```

```
ArrayTabulatedFunction.java M X
src > functions > ArrayTabulatedFunction.java > ...
7 public class ArrayTabulatedFunction implements TabulatedFunction {
210     public Iterator<FunctionPoint> iterator() {
211         return new Iterator<FunctionPoint>() {
218             public FunctionPoint next() {
220                 throw new NoSuchElementException();
221             }
222             return new FunctionPoint(points[index++]);
223         }
224     }
225     public void remove() {
226         throw new UnsupportedOperationException();
227     }
228 };
229 }
230
231 // ЗАДАНИЕ 2 - Фабрика
232 public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
233     public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
234         return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
235     }
236
237     public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
238         return new ArrayTabulatedFunction(leftX, rightX, values);
239     }
240
241     public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
242         return new ArrayTabulatedFunction(points);
243     }
244 }
245 }
246

LinkedListTabulatedFunction.java M X
src > functions > LinkedListTabulatedFunction.java > ...
10 public class LinkedListTabulatedFunction implements TabulatedFunction, Externalizable {
313     public Iterator<FunctionPoint> iterator() {
314         return new Iterator<FunctionPoint>() {
321             public FunctionPoint next() {
324                 FunctionPoint point = new FunctionPoint(node.point);
325                 node = node.next;
326                 return point;
327             }
328         }
329     }
330     public void remove() {
331         throw new UnsupportedOperationException();
332     }
333 };
334 }
335
336 // ЗАДАНИЕ 2 - Фабрика
337 public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory {
338     public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
339         return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
340     }
341
342     public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
343         return new LinkedListTabulatedFunction(leftX, rightX, values);
344     }
345
346     public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
347         return new LinkedListTabulatedFunction(points);
348     }
349 }
350 }
351 }
```



```
TabulatedFunctions.java M X
src > functions > TabulatedFunctions.java > TabulatedFunctions > setTabulatedFunctionFactory(TabulatedFunctionFactory)
1 package functions;
2
3 import java.io.*;
4
5 public class TabulatedFunctions {
6
7     // Поле фабрики. По умолчанию используем фабрику массивов.
8     private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();
9
10    private TabulatedFunctions() {}
11
12    // ЗАДАНИЕ 2 - Методы работы с фабрикой
13
14    // Метод для установки новой фабрики (смены типа создаваемых функций)
15    public static void setTabulatedFunctionFactory(TabulatedFunctionFactory newFactory) {
16        factory = newFactory;
17    }
18
19    // Создание функции через текущую фабрику (по границам и точкам)
20    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
21        return factory.createTabulatedFunction(leftX, rightX, pointsCount);
22    }
23
24    // Создание функции через текущую фабрику (по значениям)
25    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
26        return factory.createTabulatedFunction(leftX, rightX, values);
27    }
28
29    // Создание функции через текущую фабрику (по массиву точек)
30    public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
31        return factory.createTabulatedFunction(points);
32    }
33
34    // Обновленные методы
35
36    // Метод табулирования, использует текущую фабрику
37    public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) {
38        if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
39            throw new IllegalArgumentException(s: "Tabulation bounds are out of function's domain");
40        }
41        if (pointsCount < 2) {
42            throw new IllegalArgumentException(s: "Points count must be 2 or more");
43        }
44
45        FunctionPoint[] points = new FunctionPoint[pointsCount];
46        double step = (rightX - leftX) / (pointsCount - 1);
47        for (int i = 0; i < pointsCount; i++) {
48            double x = leftX + i * step;
49            points[i] = new FunctionPoint(x, function.getFunctionValue(x));
50        }
51
52        // Создаем функцию через фабрику, а не через new ArrayTabulatedFunction
53        return createTabulatedFunction(points);
54    }
55
56    // Методы ввода/вывода (также обновляем)
57
58    public static void outputTabulatedFunction(TabulatedFunction function, OutputStream out) throws IOException {
59        DataOutputStream dos = new DataOutputStream(out);
60        dos.writeInt(function.getPointsCount());
61        for (int i = 0; i < function.getPointsCount(); i++) {
62            dos.writeDouble(function.getPointX(i));
63            dos.writeDouble(function.getPointY(i));
64        }
65        dos.flush();
66    }
67
68    // Ввод из байтового потока - создает функцию через фабрику
69    public static TabulatedFunction inputTabulatedFunction(InputStream in) throws IOException {
70        DataInputStream dis = new DataInputStream(in);
71        int count = dis.readInt();
72        FunctionPoint[] points = new FunctionPoint[count];
73        for (int i = 0; i < count; i++) {
74            points[i] = new FunctionPoint(dis.readDouble(), dis.readDouble());
75        }
76        return createTabulatedFunction(points);
77    }
78
79    public static void writeTabulatedFunction(TabulatedFunction function, Writer out) throws IOException {
80        PrintWriter pw = new PrintWriter(out);
81        pw.println(function.getPointsCount());
82        for (int i = 0; i < function.getPointsCount(); i++) {
83            pw.println(function.getPointX(i) + " " + function.getPointY(i));
84        }
85        pw.flush();
86    }
87
88    // Ввод из символьного потока - создает функцию через фабрику
89    public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException {
90        StreamTokenizer tokenizer = new StreamTokenizer(in);
91    }
```

```

87
88 // Ввод из символического потока - создает функцию через фабрику
89 public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException {
90     StreamTokenizer tokenizer = new StreamTokenizer(in);
91     tokenizer.nextToken();
92     int count = (int) tokenizer.nval;
93
94     FunctionPoint[] points = new FunctionPoint[count];
95     for (int i = 0; i < count; i++) {
96         tokenizer.nextToken();
97         double x = tokenizer.nval;
98         tokenizer.nextToken();
99         double y = tokenizer.nval;
100         points[i] = new FunctionPoint(x, y);
101     }
102     return createTabulatedFunction(points);
103 }
104
105

```

Задание 3

Ещё одним инструментом, позволяющим указывать типы порождаемых объектов, является рефлексия.

В классе `TabulatedFunctions` я добавил ещё три перегруженных версии метода `createTabulatedFunction()`. Их параметры повторяют параметры трёх аналогичных методов, основанных на использовании фабрики, но также эти методы получают ссылку типа `Class` на описание класса, объект которого требуется создать. Я сделаю так, что в эти методы можно передать только ссылки на классы, реализующие интерфейс `TabulatedFunction`.

Новые методы создания объектов находят в предложенном классе конструктор с соответствующими типами параметров (например, двумя параметрами типа `double` и одним параметром типа `int` для метода, создающего объект табулированной функции по левой и правой границе области определения и количеству точек). С помощью найденного конструктора (в него должны быть переданы фактические параметры) создаётся объект табулированной функции. Ссылка на этот объект и будет возвращена из метода создания.

Если в ходе выполнения рефлексивных операций возникло исключение (не найден конструктор и т.д.), оно будет отловлено (используется блок `try` с отловом нескольких типов исключений). Вместо него будет выброшено исключение `IllegalArgumentException`, причём в его конструктор будет передано отловленное исключение из рефлексии. Это позволит в случае возникновения ошибок определить реальную причину ошибки.

В классе `TabulatedFunctions` я перегрузил методы, создающие объекты табулированных функций, добавив версии, принимающие также ссылку типа `Class` на описание класса, объект которого требуется создать. Я сделал так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс `TabulatedFunction`.

Я проверил в методе `main()` работу методов рефлексивного создания объектов, а также методов класса `TabulatedFunctions`, использующих создание объектов.


```

TabulatedFunctions.java M. Ж
src > functions > TabulatedFunctions.java > ...
1 package functions;
2
3 import java.io.*;
4 import java.lang.reflect.Constructor;
5 import java.lang.reflect.InvocationTargetException;
6
7 public class TabulatedFunctions {
8
9     // Поле фабрики. По умолчанию используем фабрику массивов.
10    private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();
11
12    private TabulatedFunctions() {}
13
14    // Методы работы с фабрикой
15
16    // Установка новой фабрики
17    public static void setTabulatedFunctionFactory(TabulatedFunctionFactory newFactory) {
18        factory = newFactory;
19    }
20
21    // Создание через фабрику (границы и количество)
22    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
23        return factory.createTabulatedFunction(leftX, rightX, pointsCount);
24    }
25
26    // Создание через фабрику (границы и значения)
27    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
28        return factory.createTabulatedFunction(leftX, rightX, values);
29    }
30
31    // Создание через фабрику (точки)
32    public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
33        return factory.createTabulatedFunction(points);
34    }
35
36    // Табулирование через текущую фабрику
37    public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) {
38        if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
39            throw new IllegalArgumentException(s: "Tabulation bounds are out of function's domain");
40        }
41        if (pointsCount < 2) {
42            throw new IllegalArgumentException(s: "Points count must be 2 or more");
43        }
44        FunctionPoint[] points = new FunctionPoint[pointsCount];
45        double step = (rightX - leftX) / (pointsCount - 1);
46        for (int i = 0; i < pointsCount; i++) {
47            double x = leftX + i * step;
48            points[i] = new FunctionPoint(x, function.getFunctionValue(x));
49        }
50        return createTabulatedFunction(points);
51    }
52
53    // ЗАДАНИЕ 3 - Методы с использованием reflection
54
55    // Создание функции указанного класса (границы и количество)
56    public static TabulatedFunction createTabulatedFunction(Class< extends TabulatedFunction> clazz, double leftX, double rightX, int pointsCount) {

```

```

TabulatedFunctions.java M. Ж
src > functions > TabulatedFunctions.java > ...
7 public class TabulatedFunctions {
8     // Создание функции указанного класса (границы и количество)
9     public static TabulatedFunction createTabulatedFunction(Class< extends TabulatedFunction> clazz, double leftX, double rightX, int pointsCount) {
10         try {
11             // Получаем конструктор с параметрами (double, double, int)
12             Constructor< extends TabulatedFunction> constructor = clazz.getConstructor(...parameterTypes: double.class, double.class, int.class);
13             // Создаем новый экземпляр
14             return constructor.newInstance(leftX, rightX, pointsCount);
15         } catch (NoSuchMethodException | InstantiationException | IllegalAccessException | InvocationTargetException e) {
16             // Оборачиваем исключения рефлексии в IllegalArgumentException
17             throw new IllegalArgumentException(e);
18         }
19     }
20
21    // Создание функции указанного класса (границы и значения)
22    public static TabulatedFunction createTabulatedFunction(Class< extends TabulatedFunction> clazz, double leftX, double rightX, double[] values) {
23        try {
24            // Получаем конструктор с параметрами (double, double, double[])
25            Constructor< extends TabulatedFunction> constructor = clazz.getConstructor(...parameterTypes: double.class, double.class, double[].class);
26            return constructor.newInstance(leftX, rightX, values);
27        } catch (NoSuchMethodException | InstantiationException | IllegalAccessException | InvocationTargetException e) {
28            throw new IllegalArgumentException(e);
29        }
30    }
31
32    // Создание функции указанного класса (массив точек)
33    public static TabulatedFunction createTabulatedFunction(Class< extends TabulatedFunction> clazz, FunctionPoint[] points) {
34        try {
35            // Получаем конструктор с параметром (FunctionPoint[])
36            Constructor< extends TabulatedFunction> constructor = clazz.getConstructor(...parameterTypes: FunctionPoint[].class);
37            // Проверяем points < Object, чтобы равнозначные аргументы newInstance работали корректно
38            return constructor.newInstance((Object) points);
39        } catch (NoSuchMethodException | InstantiationException | IllegalAccessException | InvocationTargetException e) {
40            throw new IllegalArgumentException(e);
41        }
42    }
43
44    // Табулирование с указанием класса функции
45    public static TabulatedFunction tabulate(Class< extends TabulatedFunction> clazz, Function function, double leftX, double rightX, int pointsCount) {
46        if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
47            throw new IllegalArgumentException(s: "Tabulation bounds are out of function's domain");
48        }
49        if (pointsCount < 2) {
50            throw new IllegalArgumentException(s: "Points count must be 2 or more");
51        }
52        FunctionPoint[] points = new FunctionPoint[pointsCount];
53        double step = (rightX - leftX) / (pointsCount - 1);
54        for (int i = 0; i < pointsCount; i++) {
55            double x = leftX + i * step;
56            points[i] = new FunctionPoint(x, function.getFunctionValue(x));
57        }
58        // Создаем через метод рефлексии
59        return createTabulatedFunction(clazz, points);
60    }
61
62    // Методы ввода/вывода (через фабрику)

```

```

113: public class TabulatedFunctions {
114:     // Методы ввода/вывода (через @Override)
115:
116:     public static void outputTabulatedFunction(TabulatedFunction function, OutputStream out) throws IOException {
117:         DataOutputStream dos = new DataOutputStream(out);
118:         dos.writeInt(function.getPointsCount());
119:         for (int i = 0; i < function.getPointsCount(); i++) {
120:             dos.writeDouble(function.getPointX(i));
121:             dos.writeDouble(function.getPointY(i));
122:         }
123:         dos.flush();
124:     }
125:
126:     public static TabulatedFunction inputTabulatedFunction(InputStream in) throws IOException {
127:         DataInputStream dis = new DataInputStream(in);
128:         int count = dis.readInt();
129:         FunctionPoint[] points = new FunctionPoint[count];
130:         for (int i = 0; i < count; i++) {
131:             points[i] = new FunctionPoint(dis.readDouble(), dis.readDouble());
132:         }
133:         return createTabulatedFunction(points);
134:     }
135:
136:     public static void writeTabulatedFunction(TabulatedFunction function, Writer out) throws IOException {
137:         PrintWriter pw = new PrintWriter(out);
138:         pw.println(function.getPointsCount());
139:         for (int i = 0; i < function.getPointsCount(); i++) {
140:             pw.println(function.getPointX(i) + " " + function.getPointY(i));
141:         }
142:         pw.flush();
143:     }
144:
145:     public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException {
146:         StreamTokenizer tokenizer = new StreamTokenizer(in);
147:         tokenizer.nextToken();
148:         int count = (int) tokenizer.nval;
149:
150:         FunctionPoint[] points = new FunctionPoint[count];
151:         for (int i = 0; i < count; i++) {
152:             tokenizer.nextToken();
153:             double x = tokenizer.nval;
154:             tokenizer.nextToken();
155:             double y = tokenizer.nval;
156:             points[i] = new FunctionPoint(x, y);
157:         }
158:         return createTabulatedFunction(points);
159:     }
160:
161:     // добавленные новые методы - ввод/вывод с рефлексией
162:
163:     // Считывание из байтового потока с указанием класса
164:     public static TabulatedFunction inputTabulatedFunction(Class<? extends TabulatedFunction> clazz, InputStream in) throws IOException {
165:         DataInputStream dis = new DataInputStream(in);
166:         int count = dis.readInt();
167:         FunctionPoint[] points = new FunctionPoint[count];
168:         for (int i = 0; i < count; i++) {
169:             points[i] = new FunctionPoint(dis.readDouble(), dis.readDouble());
170:         }
171:     }

```

```

144:     public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException {
145:         tokenizer.nextToken();
146:         int count = (int) tokenizer.nval;
147:
148:         FunctionPoint[] points = new FunctionPoint[count];
149:         for (int i = 0; i < count; i++) {
150:             tokenizer.nextToken();
151:             double x = tokenizer.nval;
152:             tokenizer.nextToken();
153:             double y = tokenizer.nval;
154:             points[i] = new FunctionPoint(x, y);
155:         }
156:         return createTabulatedFunction(points);
157:     }
158:
159:     // добавленные новые методы - ввод/вывод с рефлексией
160:
161:     // Считывание из байтового потока с указанием класса
162:     public static TabulatedFunction inputTabulatedFunction(Class<? extends TabulatedFunction> clazz, InputStream in) throws IOException {
163:         DataInputStream dis = new DataInputStream(in);
164:         int count = dis.readInt();
165:         FunctionPoint[] points = new FunctionPoint[count];
166:         for (int i = 0; i < count; i++) {
167:             points[i] = new FunctionPoint(dis.readDouble(), dis.readDouble());
168:         }
169:         // Используем рефлексии для создания объекта
170:         return createTabulatedFunction(clazz, points);
171:     }
172:
173:     // Считывание из символьного потока с указанием класса
174:     public static TabulatedFunction readTabulatedFunction(Class<? extends TabulatedFunction> clazz, Reader in) throws IOException {
175:         StreamTokenizer tokenizer = new StreamTokenizer(in);
176:         tokenizer.nextToken();
177:         int count = (int) tokenizer.nval;
178:
179:         FunctionPoint[] points = new FunctionPoint[count];
180:         for (int i = 0; i < count; i++) {
181:             tokenizer.nextToken();
182:             double x = tokenizer.nval;
183:             tokenizer.nextToken();
184:             double y = tokenizer.nval;
185:             points[i] = new FunctionPoint(x, y);
186:         }
187:         // Используем рефлексии для создания объекта
188:         return createTabulatedFunction(clazz, points);
189:     }
190: }
191:
192:

```

```

1  import functions.*;
2  import functions.basic.*;
3  import java.io.*;
4
5  public class Main {
6      public static void main(String[] args) {
7
8          // --- Задание 1: Итератор ---
9          System.out.println("---- Тест Итератора ----");
10         Function fCos = new Cos();
11         TabulatedFunction tFIterator = TabulatedFunctions.tabulate(fCos, leftX: 0, Math.PI, pointsCount: 5);
12         for (FunctionPoint p : tFIterator) {
13             System.out.println(p);
14         }
15
16         // --- Задание 2: Фабрика ---
17         System.out.println("---- Тест Фабрики ----");
18         Function f = new Cos();
19         TabulatedFunction tf;
20
21         tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
22         System.out.println("Фабрика по умолчанию создана: " + tf.getClass().getSimpleName());
23         functions.LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory
24         TabulatedFunctions.setTabulatedFunctionFactory(new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
25         tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
26         System.out.println("Фабрика LinkedList создана: " + tf.getClass().getSimpleName());
27
28         TabulatedFunctions.setTabulatedFunctionFactory(new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
29         tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
30         System.out.println("Фабрика Array создана: " + tf.getClass().getSimpleName());
31
32
33         // --- Задание 3: Рефлексия ---
34         System.out.println("---- Тест Рефлексии ----");
35
36         TabulatedFunction fReflectArray;
37         fReflectArray = TabulatedFunctions.createTabulatedFunction(
38             clazz: ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3);
39         System.out.println("Через рефлексию создан объект Array: " + fReflectArray.getClass().getSimpleName());
40         System.out.println(fReflectArray);
41
42         fReflectArray = TabulatedFunctions.createTabulatedFunction(
43             clazz: ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[] {0, 10});
44         System.out.println("Через рефлексию создан объект Array (по значениям): " + fReflectArray.getClass().getSimpleName());
45         System.out.println(fReflectArray);
46
47         TabulatedFunction fReflectList;
48         fReflectList = TabulatedFunctions.createTabulatedFunction(
49             clazz: LinkedListTabulatedFunction.class,
50             new FunctionPoint[] {
51                 new FunctionPoint(x: 0, y: 0),
52                 new FunctionPoint(x: 10, y: 10)
53             }
54         );
55         System.out.println("Через рефлексию создан объект LinkedList (по точкам): " + fReflectList.getClass().getSimpleName());
56         System.out.println(fReflectList);
57
58     }
59 }

```

```

58     TabulatedFunction fTabulateReflect = TabulatedFunctions.tabulate(
59         clazz: LinkedListTabulatedFunction.class, new Sin(), leftX: 0, Math.PI, pointsCount: 11);
60     System.out.println("Через рефлексию (tabulate) создан объект: " + fTabulateReflect.getClass().getSimpleName());
61     System.out.println(fTabulateReflect);
62
63     // --- Тест ввода/вывода с рефлексией (ПО ЗАМЕЧАНИЮ) ---
64     System.out.println("---- Тест ввода/вывода с рефлексией ----");
65
66     // Тест байтового потока
67     try (ByteArrayOutputStream out = new ByteArrayOutputStream();
68         TabulatedFunctions.outputTabulatedFunction(fTabulateReflect, out);
69         try (ByteArrayInputStream in = new ByteArrayInputStream(out.toByteArray());
70             // Читаем как ArrayTabulatedFunction, хотя передали LinkedList
71             TabulatedFunction fRes = TabulatedFunctions.inputTabulatedFunction(clazz: ArrayTabulatedFunction.class, in);
72             System.out.println("Байтовый поток (вводим в Array): " + fRes.getClass().getSimpleName());
73             System.out.println(fRes);
74         )
75     ) catch (IOException e) {
76         e.printStackTrace();
77     }
78
79     // Тест символьного потока
80     try (StringWriter writer = new StringWriter();
81         TabulatedFunctions.writeTabulatedFunction(fTabulateReflect, writer);
82         try (StringReader reader = new StringReader(writer.toString());
83             // Читаем как LinkedListTabulatedFunction
84             TabulatedFunction fRes = TabulatedFunctions.readTabulatedFunction(clazz: LinkedListTabulatedFunction.class, reader);
85             System.out.println("Символьный поток (читаем в LinkedList): " + fRes.getClass().getSimpleName());
86             System.out.println(fRes);
87         )
88     ) catch (IOException e) {
89         e.printStackTrace();
90     }
91 }
92 }
93

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\projects\Lab-7-2025> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:54956' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Borus\AppData\Roaming\Code\User\workspaceStorage\17aba28eb7c429ce1b9cc6313ab9a4b3\redhat.java\jdt_ws\Lab-7-2025_493eb834\bin' 'Main'
(1.5707963267948966; 6.123233995736766E-17)
(2.356194490192345; -0.7071067811865475)
(3.141592653589793; -1.0)

--- Тест Фабрик ---
Фабрика по умолчанию создала: ArrayTabulatedFunction
Фабрика LinkedList создала: LinkedListTabulatedFunction
Фабрика Array создала: ArrayTabulatedFunction

--- Тест Рефлексии ---
Через рефлексию создан объект Array: ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}
Через рефлексию создан объект Array (по значениям): ArrayTabulatedFunction
{(0.0; 0.0), (10.0; 10.0)}
Через рефлексию создан объект LinkedList (по точкам): LinkedListTabulatedFunction
{(0.0; 0.0), (10.0; 10.0)}
Через рефлексию (tabulate) создан объект: LinkedListTabulatedFunction
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0), (1.8849555921538759; 0.9510565162951536), (2.199114857512855; 0.8090169943749475), (2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.2246467991473532E-16)}

--- Тест ввода/вывода с рефлексией ---
Байтовый поток (читаем в Array): ArrayTabulatedFunction
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0), (1.8849555921538759; 0.9510565162951536), (2.199114857512855; 0.8090169943749475), (2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.2246467991473532E-16)}
Символьный поток (читаем в LinkedList): LinkedListTabulatedFunction
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.942477796076938; 0.8090169943749475), (1.2566370614359172; 0.9510565162951536), (1.5707963267948966; 1.0), (1.884955592153876; 0.9510565162951536), (2.199114857512855; 0.8090169943749475), (2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.2246467991473533)}
PS C:\projects\Lab-7-2025>

```